Lean proof gate audit — 2026-10-21

crypto repository; main `db1be50b0d4d3f9123ac1ebe9cc68538331e7bd9`

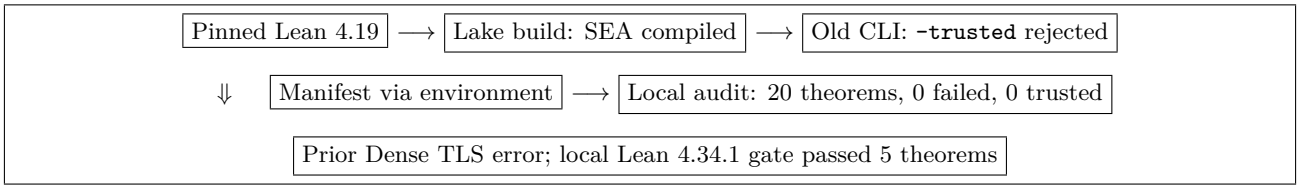
Scope. This is a validation and CI repair for the crypto repository's pinned SEA proof project and proof gate. It does not certify every historical experiment in crypto, cryptanalysis, or crypto-autoresearcher. No scientific trial or performance measurement was run.

Observed failures and correction

CI run 39459394168 at `892ef408e98f190ef19c0ab5fee8030cb5d91c6d` had 37 successful jobs, two failed jobs, and one cancelled job. These are job outcomes, not counts of verified theorems or experiments.

The SEA project built `SeaProofs.lean`, including its theorem axiom printouts, but Lean 4.19 rejected the gate's `-trusted` program argument before the separate axiom audit. This was a gate implementation failure, not a counterexample. The DenseCapacity target failed in that CI run while downloading Lean 4.34.1 because of a TLS connection error; that run produced no theorem verdict. A later local run of the repository gate passed its five theorems. The cancelled tree-split job failed during Git checkout (connection reset and `index-pack` failure), before Lean ran. The later local tree-split gate passed 24 declarations; one registered `native_decide` proof uses compiler trust and is counted separately.

The repaired gate passes the unchanged trust manifest through `LEAN_AUDIT_TRUSTED_FILE`; the auditor receives only module names as program arguments. Whitespace normalization now uses Lean APIs available in both Lean 4.19 and 4.34.1. CI installs each pinned toolchain before the proof step and retries only failed downloads, at most twice. Build and axiom failures still fail normally.



Validation and proof boundary

Check	Observed result
Auditor source under Lean 4.19.0 and 4.34.1	Both compile. This is a compatibility check, not a theorem audit.
SEA project proof gate	PASS; 20 theorem declarations, 0 failed, 0 compiler-trusted. The count includes generated GroupOps declarations.
DenseCapacity proof gate	PASS; 5 arithmetic-bound theorems, 0 failed, 0 compiler-trusted.
Tree-split proof gate	PASS; 24 declarations, 0 failed, 1 manifest-registered compiler-trusted `native_decide` proof.
Gate self-test	Seven of seven expected positive and negative controls passed.
Lean 4.19 controls	Custom axiom rejected; registered native proof reported as one trusted proof.
Rust library tests	PASS: 4,464 passed, 0 failed, 119 ignored. The compressed raw log is retained with the report.

The SEA declarations include `SeaProofs.bsgs_complete`, a search completeness theorem for an explicitly supplied additive group, and `SeaProofs.direct_equals_full_intersection`, a predicate identity. The local audit found only standard Lean foundations, `propext` and `Quot.sound`, for SEA. It does not prove that the Rust point-counting implementation conforms to the abstract predicates or establish a new speed or ECDLP claim. The older successful CI jobs are not a fresh build of current main; the PR's full proof matrix remains a separate gate. The complete Rust test used the minimal sparse profile with the SEA fixture and local `isogeny_algos` dependency included. Two earlier sparse checkout attempts lacked tracked inputs and failed during compilation; their logs are retained. The successful raw test log SHA-256 is `de7281263c07499c30f671e7e0821010f60a16f918095b18c2ff6a37ebf95271`.

Requested scope and graphs

Requirement	Status in this change
Faithful Lean proofs for all prior experiments across three repos	Partial: this repair restores one proof project's axiom gate; other source-to-claim reviews and lemmas remain.
Code and numerical result validation	Partial: gate, SEA model, DenseCapacity bounds, tree-split target, and Rust library tested; historical results not replayed.
Performance graphs and dashboards	Not attempted: no new performance datum or correction. Canonical performance figures should not move for a CI repair.
All results good to go	Not established: full crypto proof matrix and cross-repository semantic reviews remain. The separate autoresearcher workspace build passed.

The separate `diagram.svg` depicts observed gate states; it is not a performance graph. The companion Markdown report gives exact reproduction commands, retained logs, and audited source SHA-256 values for the SEA, DenseCapacity, and tree-split targets.